

Informatica per le Biotecnologie

Algoritmica Lezione 4

1. Il problema dell'ordinamento di dati

sorting in gergo informatico, consistente nel disporre in ordine “crescente” n elementi tra cui è definita una relazione di ordinamento totale indicata con $\leq$.

Gli elementi si memorizzano in un vettore $A[0 \dots n-1]$

L'algoritmo iterativo INSERTION-SORT si basa sull'ipotesi che i primi i elementi contenuti tra le posizioni 0 e $i - 1$ siano ordinati: si inserisce l'elemento $A[i]$ tra questi e si ripete l'operazione a partire da $i + 1$ finché tutti gli n elementi sono stati inseriti.

↓
3 8 5 2 13 10 . . .

3 8 5 2 13 10 . . .



3	8	5	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---



3	5	8	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	8	5	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---



3	5	8	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	8	5	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	5	8	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---



3	5	2	8	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	8	5	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---



3	5	8	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---



3	2	5	8	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	8	5	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	5	8	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---



2	3	5	8	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	8	5	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	5	8	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---



2	3	5	8	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	8	5	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---

3	5	8	2	13	10	.	.	.
---	---	---	---	----	----	---	---	---

2	3	5	8	13	10	.	.	.
---	---	---	---	----	----	---	---	---



2	3	5	8	10	13	.	.	.
---	---	---	---	----	----	---	---	---

INSERTION-SORT(A)

```
for ( $i$  = 1;  $i \leq n - 1$ ;  $i++$ )  
    {  $k$  =  $A[i]$ ;  
       $j$  =  $i - 1$ ;  
      while ( $(j \geq 0) \ \&\& \ (A[j] > k)$ )  
          {  $A[j + 1] = A[j]$ ;  $j = j - 1$  };  
       $A[j + 1] = k$ ; }
```

Prima iterazione del **for**

j	i								k = 8
3	8	5	2	13	10	.	.	.	

INSERTION-SORT(A)

```
for ( $i$  = 1;  $i \leq n - 1$ ;  $i++$ )  
    {  $k$  =  $A[i]$ ;  
       $j$  =  $i - 1$ ;  
      while ( $(j \geq 0) \ \&\& \ (A[j] > k)$ )  
          {  $A[j + 1] = A[j]$ ;  $j = j - 1$  };  
       $A[j + 1] = k$ ; }
```

Primi tre assegnamenti nella seconda iterazione del **for**

	j	i							$k = 5$
	3	8	5	2	13	10	.	.	.

INSERTION-SORT(A)

```
for (i = 1; i ≤ n - 1; i++)
```

$$\{ \underline{k} = A[i];$$
$$\underline{j = i - 1;}$$

```
while ((j ≥ 0) && (A[j] > k))
```

$$\{A[j+1] = A[j]; j = j - 1\};$$
$$A[j + 1] = k; \}$$

Effetto della seconda iterazione del **for**

j **i** **k = 5**

3 **5** **8** 2 13 10 . . .

INSERTION-SORT(A)

for (i = 1; $i \leq n - 1$; $i++$)

{ k = $A[i]$;

j = $i - 1$;

while ($(j \geq 0) \ \&\& \ (A[j] > k)$)

 { $A[j + 1] = A[j]$; $j = j - 1$ };

$A[j + 1] = k$; }

Primi tre assegnamenti nella terza iterazione del **for**

j i

k = 2

3 5 8 2 13 10 . . .

Caso ottimo: i dati in ingresso sono ordinati
in modo crescente

2 3 5 8 10 13 20 23

Il ciclo while non è mai eseguito :

il tempo $T(n)$ è di ordine $\Theta(n)$ (numero di iterazioni di i)

Caso pessimo: i dati sono ordinati in modo
decrescente

23 20 13 10 8 5 3 2

Il ciclo while richiede $1 + 2 + 3 + \dots + n-1$ operazioni nei
successivi cicli di i : il tempo $T(n)$ è di ordine $\Theta(n^2)$

Dunque **INSERTION-SORT** richiede tempo $O(n^2)$
considerando il caso pessimo

Un algoritmo **ricorsivo** di ordinamento

MERGE-SORT: ordinamento per fusione

Oltre al vettore A che contiene i dati che vengono riordinati man mano, è necessario un vettore di appoggio $B[0...n-1]$

MERGE-SORT(A, s, d) $s \leq d$

// Ordinamento di A tra le posizioni s,d //

if ($s < d$) {

$m = \lfloor (s + d) / 2 \rfloor$;

MERGE-SORT(A, s, m);

MERGE-SORT($A, m + 1, d$);

FUSIONE(A, s, m, d); }

Chiamata dall'esterno: MERGE-SORT($A, 0, n-1$)

Le due chiamate interne di MERGE-SORT ordinano le due metà del vettore A. La procedura FUSIONE le fonde in un unico vettore B e poi le trasferisce in A.

MERGE-SORT(A, s, d) $s \leq d$

// Ordinamento di A tra le posizioni s,d //

if ($s < d$) {

$m = \lfloor (s + d)/2 \rfloor$;

MERGE-SORT(A, s, m);

MERGE-SORT($A, m + 1, d$);

FUSIONE(A, s, m, d); }

Tutte le righe del programma, fino alle due chiamate ricorsive, hanno lo scopo di stabilire l'ordine in cui si fanno i confronti. Questi sono tutti eseguiti dalla procedura FUSIONE. Come funziona FUSIONE ?

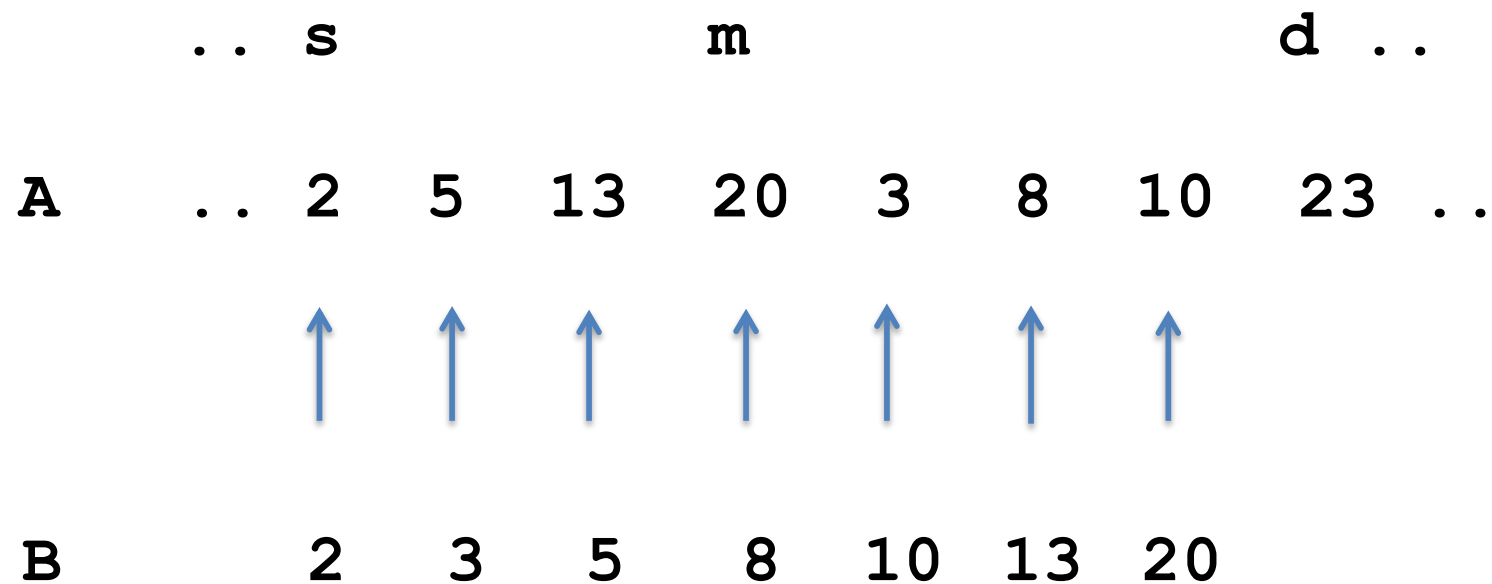
	..	s			m				d	..
A	..	<u>2</u>	5	13	20	<u>3</u>	8	10	23	..
B		2								

	..	s			m				d	..
A	..	2	<u>5</u>	13	20	<u>3</u>	8	10	23	..
B		2	3							

	..	s			m				d	..
A	..	2	<u>5</u>	13	20	3	<u>8</u>	10	23	..
B		2	3							

	..	s			m				d	..
A	..	2	<u>5</u>	13	20	3	<u>8</u>	10	23	..
B		2	3	5						

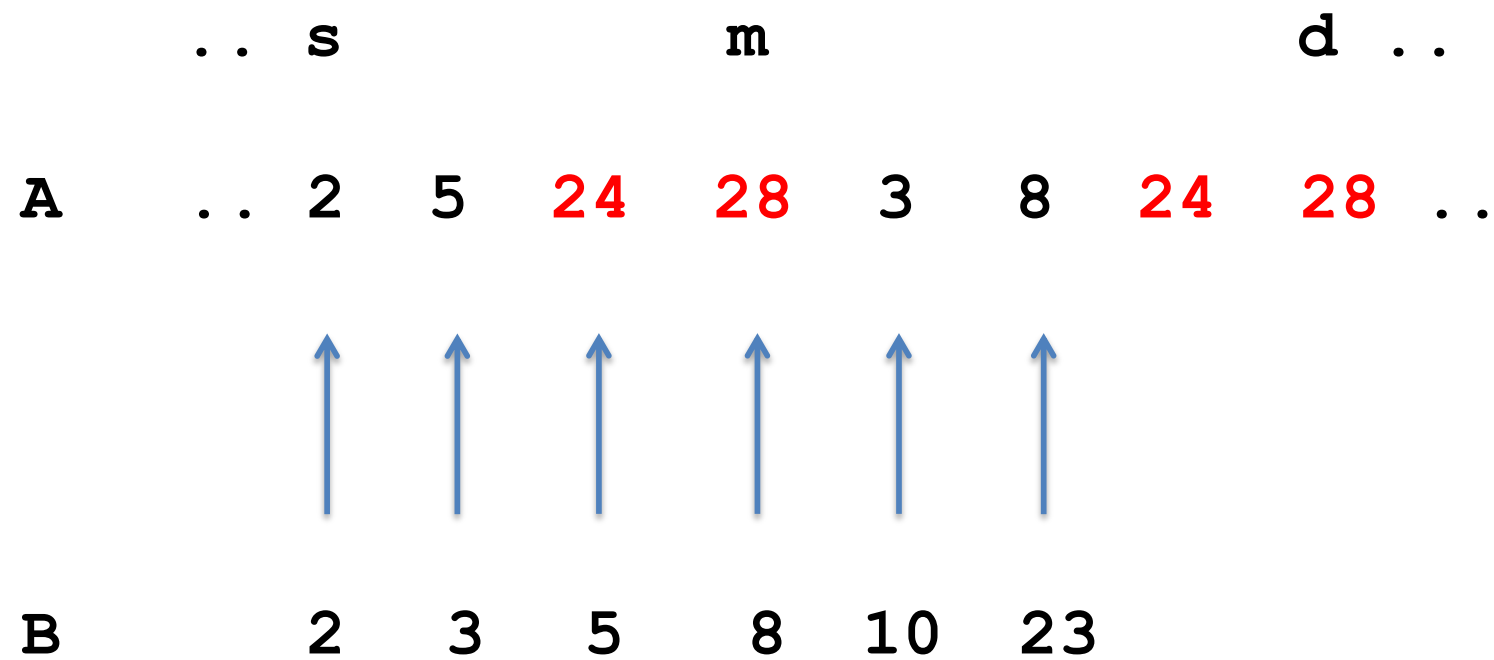
	..	s			m				d	..
A	..	2	5	<u>13</u>	20	3	<u>8</u>	10	23	..
B		2	3	5	.	.	.	.	.	.
B		2	3	5	8	10	13	20		



Ho ordinato $A[s:d]$.

Il sottovettore $A[s:m]$ si è esaurito prima di $A[m+1:d]$: ma se avviene il contrario?





FUSIONE(A, s, m, d)

$i = s; j = m + 1; k = 0;$

while $((i \leq m) \ \&\& \ (j \leq d)) \ \{$

if $(A[i] \leq A[j]) \ \{B[k] = A[i]; \ i = i + 1\}$

else $\{B[k] = A[j]; \ j = j + 1\};$

$k = k + 1 \ \};$

// se il sottovettore $A[m + 1 : d]$ è esaurito //

if $(i \leq m)$

$\{ j = d - (m - i);$

for $(\ ; i \leq m; i ++; j ++) \ A[j] = A[i] \ \};$

// si portano in $A[s...]$ i k elementi di B //

$i = s; j = 0;$

for $(\ ; j \leq k - 1; i ++; j ++) \ A[i] = B[j];$

Ordine dei confronti di FUSIONE entro MERGE-SORT

2 20 13 5 8 23 10 3 6 9 21 12 7 16 19 4

2 20

5 13

2 5 13 20

8 23

3 10

3 8 10 23

2 3 5 8 10 13 20 23

6 9

.....

Numero massimo di confronti di FUSIONE, $n = 16$

2 | 20 13 | 5 8 | 23 10 | 3 6 | 9 21 | 12 7 | 16 19 | 4 $n-8$

2 20 | 5 13 8 23 | 3 10 6 9 | 12 21 7 16 | 4 19 $n-4$

2 5 13 20 | 3 8 10 23 6 9 12 21 | 4 7 16 19 $n-2$

2 3 5 8 10 13 20 23 | 4 6 7 9 12 16 19 21 $n-1$

2 3 4 5 6 7 8 9 10 12 13 16 19 20 21 23

Per $n = 16 = 2^4$, FUSIONE ha eseguito al massimo
 $4n-1-2-4-8 = n \log_2 n - (n-1) = \Theta(n \log n)$ confronti

Il tempo richiesto è maggiore se l'esame del
sottovettore $A[m+1:d]$ termina prima di
quello di $A[s:m]$, ma in entrambi i casi tale
tempo è di ordine $\Theta(d-s)$, quindi è **di ordine**
 $\Theta(n)$ quando lavora sull'intero vettore.

MERGE-SORT(A, s, d) $s \leq d$

// Ordinamento di A tra le posizioni s,d //

if ($s < d$) {

$m = \lfloor (s + d)/2 \rfloor$;

MERGE-SORT(A, s, m);

MERGE-SORT($A, m + 1, d$);

FUSIONE(A, s, m, d); }

Tempo $T(n)$ richiesto da MERGE-SORT

- per $n = 1$ (cioè per $s = d$) abbiamo $T(1) = b$ costante
- per $n > 1$ (cioè per $s < d$) la procedura richiede tempo costante c_1 fino al calcolo di m ; chiama due volte se stessa su $n/2$ dati; esegue FUSIONE in tempo $c_2 n$. Dunque:
 $T(n) = 2T(n/2) + c_1 + c_2 n.$ c_1 e c_2 costanti

Abbiamo *l'equazione di ricorrenza*:

$$T(1) = b; \quad T(n) \leq 2T(n/2) + cn, \quad \text{per } n > 1;$$

$$c = c_1 + c_2 \text{ costante}$$

Sviluppando l'equazione si trova

$T(n)$ di ordine $\Theta(n \log n)$ (Vedi Dispensa)

Un **limite inferiore** alla complessità del sorting

Calcoliamo il **numero minimo di confronti** tra elementi dell'insieme da ordinare, impiegando un albero di decisione

Ogni **permutazione** della sequenza iniziale dei dati è una possibile soluzione del problema, quindi il numero $S(n)$ di soluzioni è ***n fattoriale***

Applicando la Formula di Stirling: $S(n) = n! \approx \sqrt{2\pi n}(n/e)^n$

Allocando queste soluzioni nelle foglie di un albero ternario con percorsi di computazione di **lunghezza massima t** abbiamo $3^t \geq \sqrt{2\pi n}(n/e)^n$

Applicando il logaritmo a base 3 ai due membri della disuguaglianza otteniamo:

$$t \geq (1/2) \log_3(2\pi n) + n(\log_3 n - \log_3 e) > n(\log_3 n - \log_3 e)$$

dunque **$\min(n) = t$** è di ordine **$\Omega(n \log n)$** .

Poiché un **limite inferiore** alla complessità di tempo del problema di sorting è $\Omega(n \log n)$ l'algoritmo MERGESORT che richiede tempo $\Theta(n \log n)$ ed è **ottimo** mentre INSERTION-SORT non lo è.

Ricordiamo tuttavia che MERGE-SORT richiede uno spazio doppio di memoria a causa dell'impiego (necessario) del vettore B mentre INSERTION-SORT è eseguito solo con scambi all'interno del vettore A.

Vi sono altri algoritmi di sorting che richiedono tempo $\Theta(n \log n)$ e non richiedono un vettore ausiliario

2. Algoritmi polinomiali e esponenziali

MERGE-SORT e INSERTION-SORT richiedono tempi $\Theta(n \log n)$ e $\Theta(n^2)$ nel caso pessimo. Indichiamoli con A_1 e A_2 .

Confrontiamoli con l'algoritmo FOOLISH-SORT, indicato con A_3 , che genera tutte le permutazioni dei dati d'ingresso, controllandole una a una fino a individuare quella ordinata.

FOOLISH-SORT richiede almeno tempo $n!$ per generare tutte le permutazioni (nel caso pessimo la permutazione ordinata è l'ultima generata), e tempo n per verificare l'ordinamento in ciascuna di esse. Complessivamente tempo $n \times n!$

Ricordando che: $n! \approx \sqrt{2\pi n}(n/e)^n$,

il tempo complessivo è $\Theta(n^{3/2}(n/e)^n)$.

Un algoritmo è detto **polinomiale** se la sua complessità in tempo è **limitata superiormente** da un polinomio in n , ovvero è di ordine $O(n^k)$ per $k > 0$ costante.

Un algoritmo è detto **esponenziale** se la sua complessità in tempo è **limitata inferiormente** da una funzione che presenta n all'esponente, ovvero è di ordine $\Omega(k^f)$ per $k > 0$ costante e f crescente con n .

Tempo di A_1 (MERGE-SORT) $\Theta(n \log n)$

Tempo di A_2 (INSERTION-SORT) $\Theta(n^2)$

Tempo di A_3 (FOOLISH-SORT) $\Theta(n^{3/2} (n/e)^n)$

A_1 e A_2 sono algoritmi *polinomiali* (in particolare il tempo di A_1 è limitato superiormente da $n^{1+\epsilon}$ per qualsiasi $\epsilon > 0$ poiché $n^\epsilon > \log n$).

A_3 è un algoritmo *esponenziale* perché nell'espressione del tempo n figura (anche) all'esponente.

Poniamo che i tre tempi siano **espressi in secondi** se gli algoritmi sono codificati in un particolare linguaggio su un particolare calcolatore

$$T_1 = \frac{1}{100}n \log_2 n, \quad T_2 = \frac{1}{100}n^2, \quad T_3 = \frac{1}{100}n^{3/2}(n/e)^n$$

n	4	8	16	32	64	128
T_1	0,08	0,24	0,64	1,60	3,84	8,96
T_2	0,16	0,64	2,56	10,24	40,96	163,84
T_3	0,38	$>10^3$	$>10^{12}$	$>10^{37}$	$>10^{91}$	$>10^{215}$

Al raddoppiare di n il tempo T_1 è poco più che doppio

$$T_1(2n) = \frac{1}{100} 2n \log_2(2n) = \frac{2}{100} n (\log_2 n + 1) = \frac{2}{100} n \log_2 n + \frac{2}{100} n$$

$$T_1(2n) = 2T_1(n) + \frac{2}{100} n$$

Al raddoppiare di n il tempo T_2 è quattro volte tanto

$$T_2(2n) = \frac{1}{100} (2n)^2 = \frac{4}{100} n^2 = 4T_2(n).$$

Con la crescita esponenziale, per semplificare i calcoli consideriamo un nuovo algoritmo A_4 di complessità “solo” 2^n anziché $(n/e)^n$

$$T_4(n) = \frac{1}{100} 2^n$$

$$\text{dunque } T_4(2n) = \frac{1}{100} 2^{2n} = \frac{1}{100} (2^n)^2$$

Per ordinare 128 elementi impiegando A_4 occorrono più di 10^{39} secondi (10^{29} millenni).

Una nuova considerazione:

Per un algoritmo arbitrario A eseguito su un calcolatore C , impieghiamo un calcolatore C' di velocità k volte maggiore di quella di C e vediamo come cresce la dimensione dei dati trattabili, n e n' , a pari tempo τ di elaborazione.

In prima approssimazione impiegare C' per τ secondi equivale a impiegare C per $k\tau$ secondi:

Se A richiede tempo lineare, $T(n) = cn$, abbiamo

$$cn = \tau, \quad cn' = k\tau \quad \text{quindi} \quad n' = kn$$

cioè la dimensione dei dati è *k volte maggiore*.

Calcoliamo cosa accade per gli algoritmi
di ordinamento A_1, A_2, A_4

Per A_1 abbiamo: $\frac{1}{100}n \log_2 n = \tau, \quad \frac{1}{100}n' \log_2 n' = k\tau,$

da cui $n' \log_2 n' = kn \log_2 n$: poiché $\log_2 n'$ è poco maggiore di $\log_2 n$, i valori di n' sono quasi k volte i valori di n .

Poniamo $k = 10$. Per $\tau = 8,96$ secondi, con A_1 si possono ordinare $n = 128$ elementi su C (vedi tabella precedente) e $n' = 911$ su C' , con incremento poco meno di $k = 10$

Per A_2 abbiamo:

$$\frac{1}{100}n^2 = \tau, \quad \frac{1}{100}n'^2 = k\tau,$$

da cui $n'^2 = kn^2$ ovvero $n' = \sqrt{k} n$

per $k = 10$ abbiamo $n' = 3,16 n$

Con un algoritmo di complessità n^c , con $c > 2$, avremmo $n' = k^{1/c} n$, con un vantaggio tanto minore quanto più alto è il valore di c , cioè quanto meno efficiente è l'algoritmo.

Per un algoritmo esponenziale per esempio A_4 (crescita 2^n):

$$\frac{1}{100}2^n = \tau, \quad \frac{1}{100}2^{n'} = k\tau,$$

da cui $2^{n'} = k2^n$, quindi $n' = n + \log_2 k$

Per esempio moltiplicando per $k = 1024$ la velocità del calcolatore, a pari tempo τ , con A_4 si potrebbero ordinare

$$n' = n + \log_2 1024 = n + 10 \text{ dati.}$$

Gli algoritmi esponenziali sono inutilizzabili indipendentemente dalla velocità dei calcolatori impiegati

3. Complessità computazionale

Esistono problemi che richiedono necessariamente algoritmi esponenziali per risolverli: possono essere **banali** (per esempio “stampare tutte le permutazioni di un insieme”) o comunque non essere importanti in pratica.

Ma, come vedremo sotto, *si incontrano molti problemi importanti e non banali* che richiedono tempo esponenziale perché non siamo capaci di risolverli meglio.

Se in un problema che richiede tempo esponenziale la correttezza di una soluzione può essere **verificata** (ma non trovata) in tempo polinomiale, tale soluzione è detta ***certificato***.

Dunque un certificato rappresenta una possibile soluzione che però non è nota a priori.

Per quanto sappiamo oggi, per **risolvere** uno di questi problemi è necessario enumerare e controllare un numero esponenziale di possibili configurazioni fino a trovare quella che rappresenta una valida soluzione.

Informalmente (perché la faccenda è piuttosto complicata):

Definizione:

P è l'insieme dei problemi **risolubili** in tempo polinomiale.

NP è l'insieme dei problemi **verificabili** in tempo polinomiale.

Per un problema in P si può invece verificare in tempo polinomiale l'esistenza di una soluzione risolvendolo direttamente. Da ciò segue la relazione fondamentale:

$$P \subseteq NP.$$

Non è mai stato dimostrato se vale il contenimento stretto tra le due classi, cioè se P è diverso da NP o se esiste un algoritmo polinomiale per risolvere i problemi in NP .

Questo quesito è considerato uno dei più importanti problemi irrisolti della matematica.

Esempi di problemi in $\mathcal{NP}$

1. **Biologia molecolare:** dato un insieme arbitrario di sequenze di DNA stabilire se esiste una sottosequenza comune di lunghezza assegnata.
2. **Manifattura:** stabilire se un insieme di pezzi di lamiera possono essere ottenuti tagliandoli da un foglio assegnato disponendoli opportunamente su di esso.
3. **Scheduling:** dato un insieme di procedure da eseguire su un insieme di macchine, stabilire se tali procedure possono essere distribuite tra le macchine in modo che tutte queste terminino di operare entro un tempo prefissato.

4. Algebra: stabilire se un'equazione algebrica di secondo grado in due variabili x, y , a coefficienti interi, ammette una soluzione in cui x e y hanno valori interi (se l'equazione è di primo grado il problema è in $\mathcal{P}$).

Questi problemi si fanno risolvere solo provando un numero esponenziale di possibili soluzioni. Si può però verificare la correttezza di una soluzione in tempo polinomiale.